

C_Course :: Day 2

Operation

Branching

Looping

* Operation

* uni operation

→ Increment ++
short int no = 50;

→ No ++;

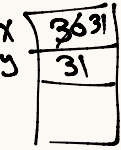
→ decrement --
No --;

uni operation

Pre Increment

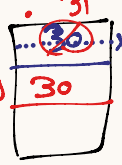
char x = 30;
char y = 0;

y = ++x;



Post Increment

char x = 30;
char y = 0;
y = x++;



* Mathematical operation → 2 operand.

→ + → - → * → / → % Remainder

* Assignment operation (=)

LHS = RHS;

Variable
Location in Memory
Can be Modified

Notes: ① if LHS is Not var (const value) → error L Value

ex: char sum = 0;

sum = sum + 3; ⇒ sum += 3;

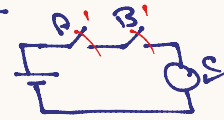
sum = sum - 4; ⇒ sum -= 4;

* Bit wise operation

* logic gates

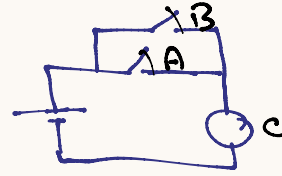
* And gate

A	B	C
0	0	0
0	1	0
1	0	0
1	1	1



* OR gate

A	B	C
0	0	0
0	1	1
1	0	1
1	1	1



* Not gate

A	B
0	1
1	0



* XOR gate

A	B	C
0	0	0
0	1	1
1	0	1
1	1	0

* Bitwise operation

① And (&) ② OR (|)

③ Complement (~) ④ Shift

→ Right (>>)

→ Left (<<)

ex: char x = 5; → 0000 0101
char y = 7; → 0000 0111
code char z = x & y; →

x	00000101
y	00000111
z	00000101

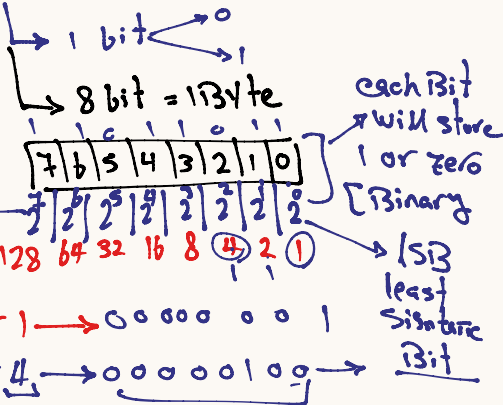
x → 0000 0101
y → 0000 0111
z → 0000 0101

ex: char x = 13; → 0000 1101
char y = 30; → 0001 1110
char z = x | y; → 0001 1111
Print("z" → 31)

x	00001101
y	00011110
z	00011111

Bit wise operation

→ Revision about storage



Short int y
LSB: 0...2...15
MSB: 15...2...0

ex: char x = 20;

z = ~x;

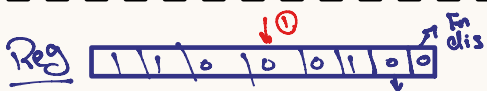


left shift <<

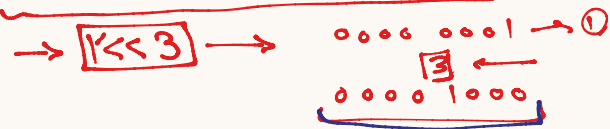
var << No of Shift
X << 1
① Remove MSB
② Move the bit to left
③ write 0 in LSB

* Right Shift: >>

var >> No of shifting
X >> 1
① LSB Remove
② Moving the bits
③ In MSB write zero



① → write one in specific bit with don't change another bit



* Set Bit → write 1 in specific bit

Assume → Reg → No



Reg $\oplus (1 \ll 3)$

Reg = Reg $\mid (1 \ll 3)$

① left shift Number 1 to Bit No

② Result from step 1 or with Reg

③ Store the Result in Reg

set Bit →

$$\text{Reg} = \text{Reg} \mid (1 \ll \text{BitNo})$$

* clear Bit → write 0 in specific Bit



Reg:

① → $1 \ll \text{BitNo}$

$1 \ll 5$

$0 0 1 0 0 0 0 0$

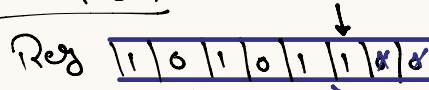
Reg $\& \sim (1 \ll 5)$

② Result complement $\sim (1 \ll 5)$

clear Bit

$$\text{Reg} = \text{Reg} \& \sim (1 \ll \text{BitNo});$$

Read Bit



Reg $\gg 2$



$\& 1$

Result = $((\text{Reg} \gg \text{BitNo}) \& 1)$ correct

Reg = $((\text{Reg} \gg \text{BitNo}) \& 1)$ error

toggle Bit → change state for specific Bit

$$\text{Reg} = \text{Reg} \wedge (1 \ll \text{BitNo})$$

* Conditional operation

* greater than → >
 less than <
 greater than or eq ≥
 less than or eq ≤
 eq ==
 Not eq !=

* Condition Result

True

any Number without 0

(1)
2
3

False
0

Logical operation

Condition 1 && Condition 2

Condition 1	Condition 2	R
False	False	False
False	True	False
True	False	False
True	True	True

Condition 2 || Condition 1

Condition 2	Condition 1	R
F	F	F
F	T	T
T	F	T
T	T	T

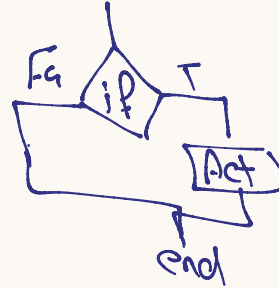
* Branching [Conditional Statement]

if statement

switch statement

if statement: check Condition
 True
 False

① syntax ① → if (Condition)
 {
 Action;
 }



ex: char degree = 0;
 scanf("%d", °);
 if (degree >= 50)
 {
 printf("Pass");
 }

② syntax 2
 if....else if

Condition
 * unlimited
 else if
 * Must else if
 have if

if (Condition)
 {
 Action 1;
 }

else if (Condition)
 {
 Action 2;
 }

else if (Condition)
 {
 Action 3;
 }

* Don't write
 code between
 if...else if

if (Condition)
 {
 Action 1;
 }

* char x = 30;
 else if (Condition)
 {
 }

else if
 without
 if

③ syntax 3 → temp > 25

if (Condition)
 {
 Action
 Action Air Condition on
 }
 else
 {
 Air Condition off
 }

* we have one else only

* write code scan degree from

User → 0: 100

Fail → 0: 49

Pass → 50: 70

Good → 70: 80

V. Good → 80: 90

excellent → 90: 100

less than 0

Not valid

More than 100

Not valid

* Switch Case

↳ ① check equality

③
switch (variable)
{

Case 3 : Action₁; break;
Case 1 : Action₂; break;
Case 4
Case 6
Case 10

default :

}

switch (Plat No)

{
default : printf("Not Valid No"); break;
Case 7 : printf("CR7"); break;
Case 10 : printf("Eizo"); break;
Case 19 : printf("Abelallah Said");

* Case Can be Non orderd

* Unlimited From Cases.

* we have default Case

↳ one only [like else]
↳ optional

* we need key word to
exit from switch case

[break]

* Scan From user the ID

* by Switch Case

3241 → welcome Ahmed
3392 → welcome Hesham
4501 → welcome Omar
any Number → wrong ID.

* looping :

* for loop → when know number of
Iteration (

for (start ; Condition ; steps)
{
Action

ex: unsigned char Count = 0;
for (Count = 0 ; Count < 10 ; Count++)

{
③ printf(" %d\n", Count);

}

start
Condition
Action
steps

Condition

write C-Code to get 10 number and Print
Sum & Avg.

#include <stdio.h>

int main ()

{
unsigned char number = 0;

unsigned char Count = 0;

int sum = 0;

for (Count = 0 ; Count < 10 ; Count++)

{
scanf(" %d", &Number);

SUM = SUM + number ;

float Avg = (float) SUM / 10 ;

printf(" SUM = %d\n", SUM);

printf(" Avg = %f\n", Avg);

}

✓ while loop → still Repeat untill event
 Condition → False
 while (Condition) { while (Phone != "Recive Call")
 { Action; { Run(); }
 } }

Time False

do..... while

do {
 Action
 } while (Condition);

unsigned char count = 0; → Infinite loop
 for (count = 0; count < 256; count++)
 { printf(" %d\n", count);
 }

→ 255 → check 255 < 256
 Count 256 + 1
 255 {
 0000 0001 +
 0000 0000